NONPROFIT SUCCESS AI PRODUCT REQUIREMENTS DOCUMENT

Implementation Review Edition
Three Workstreams: UI/UX • CRM & Data Engineering • Knowledge/RAG, Agent Memory & Deployment

1. Executive Summary

This PRD converts the revised Success Portal AI system design into three implementation workstreams. The product is a governed multi-agent platform, not a collection of isolated AI agents. Scout handles qualification/routing; Architect handles assessment/planning; Chronicle handles impact synthesis and institutional learning. Shared deterministic services own tasks, calendar, communications, contracts, approvals, and other state-changing capabilities.

The recommended foundation is Supabase as the shared identity and operational data layer for GrantPilot and Success Portal AI. The implementation uses Option B for contracts: preview and sign in-app, with the verified signature event gating the lifecycle transition.

2. Product Principles

- Every capability has one owner, one system of record, and one audit trail.
- Agents reason; deterministic backend services execute side effects.
- Skills, plugins, and MCPs are shared platform resources, not agent-specific implementations.
- AI outputs are structured, validated, permission-checked, and human-approved where consequential.
- Quality is a release requirement: deterministic tests, golden sets, LLM-as-a-judge, adversarial testing, human acceptance, and production monitoring.
- Start with governed RAG and structured memory; defer a knowledge graph until justified by real use cases.
- Converge GrantPilot and Success Portal on one Supabase identity/data foundation.

3. Target System Boundary

- Applications: GrantPilot and Success Portal AI.
- Identity/data: one canonical Supabase project with Supabase Auth, Postgres, RLS, storage, and database functions.
- Agent runtime: Scout, Architect, Chronicle using shared policies, schemas, skills, plugins, MCP catalog, model gateway, and observability.
- Execution boundary: agents produce typed intents/recommendations; authorized services perform writes and external side effects.
- Knowledge: approved document store + vector retrieval + provenance + scoped memory; knowledge graph deferred.
- Quality: shared evaluation harness across agents, tools, retrieval, and workflows.
- Deployment: server-side AI/runtime services; no privileged model/tool credentials in the browser.

4. Workstream 1 — UI & UX: TONY

4.1 User Journeys

- Shared Supabase login → organization/profile context → role-aware dashboard.
- Scout queue → qualification evidence → recommendation → approve/request changes/reject.

- Approved intake → Architect assessment → priorities → scoped plan → canonical tasks/resources → contract preview.
- Option B contract flow: preview → authorized in-app signature → immutable signature event → lifecycle transition.
- Engagement operations: canonical tasks, milestones, calendar, communications, and status.
- Chronicle: completed engagement → impact summary → lessons → candidate knowledge → human review/publish.
- AI oversight: provenance, evidence, tool activity, approvals, and relevant evaluation status.

4.2 UI Requirements

- Role-based navigation and route protection using Supabase identity and application roles.
- Consistent entity model for organizations, engagements, assessments, tasks, contracts, documents, and users.
- AI-generated content must be visually distinguishable from verified source data.
- Consequential recommendations expose evidence before approval.
- Approval actions show the exact side effect before confirmation.
- Contract signing shows final artifact, signer identity, timestamp, and status.
- Task/calendar views consume canonical backend records; no agent-local task stores.
- Errors identify validation, permission, integration, or AI failures.
- Accessibility, responsive behavior, keyboard navigation, and loading/empty/error states are release requirements.

4.3 UX Acceptance Criteria

- A GrantPilot user can sign in with the same account and reach the correct organization/role context.
- A reviewer can approve, reject, or request changes without leaving the relevant workflow.
- No UI action bypasses backend authorization.
- A contract cannot advance stages unless the backend verifies a valid signature event.
- Users can distinguish AI recommendations, retrieved evidence, and human-verified facts.
- Critical actions have visible success/failure states and an audit reference.

5. Workstream 2 — Backend CRM & Data Engineering: Karthik

5.1 Architecture Decision

Use Supabase as the shared identity and operational data foundation. Because GrantPilot already uses Supabase and both products need the same login/user base, do not expand the Firebase production model. Migrate Success Portal to the same Supabase project before substantial production data accumulates.

5.2 Canonical Entities

- auth.users — canonical identity.
- profiles — application profile.
- organizations — nonprofit/partner organizations.
- organization_members — membership and roles.
- intakes — submitted requests.
- scout_reviews — qualification evidence, recommendation, model/version, reviewer decision.
- assessments — Architect outputs and human edits.
- engagements — lifecycle record.
- tasks — canonical task system of record.
- milestones — canonical engagement milestones.
- contracts — generated/final artifacts.
- signatures — immutable signature events.
- documents — source/generated document metadata and provenance.
- communications — operational communications.
- calendar_events — external calendar references.
- agent_runs — agent execution metadata.

- tool_calls — tool/MCP audit metadata.
- approvals — human decisions.
- audit_events — immutable operational/security trail.

5.3 Data Engineering Requirements

- Postgres constraints and typed schemas are the first correctness layer.
- Use Row Level Security for tenant and role boundaries.
- Every business entity has explicit organization ownership.
- Separate operational data from AI telemetry/evaluation datasets.
- Version prompts, models, policies, and schemas.
- Database migrations live in source control; no manual schema drift.
- Use idempotency for workflow writes and external side effects.
- Use an outbox/event pattern where reliable external delivery is required.
- Maintain Firebase→Supabase identity mapping during migration and reconcile before cutover.
- Define retention, deletion, export, and audit requirements.

5.4 CRM Lifecycle

Intake → Scout review → approved opportunity → Architect assessment → engagement → contract/signature → delivery → Chronicle impact review → institutional learning.

5.5 Integration Ownership

Calendar, email, CRM synchronization, events, marketing, and other external systems are shared platform capabilities. Agents invoke approved operations through shared skills/plugins/MCPs; they do not own separate implementations.

6. Workstream 3 — Knowledge Base, RAG, Agent Memory & Deployment

6.1 Shared Multi-Agent Platform

- Skills Registry: versioned reusable business capabilities with schemas, permissions, preconditions, and tests.
- Plugin Registry: packaged external integrations behind consistent interfaces.
- MCP Catalog/Gateway: governed MCP servers/tools with authentication, scopes, logging, and rate limits.
- Policy Registry: approval, safety, data-access, and side-effect rules.
- Schema Registry: shared typed agent/tool contracts.
- Model Gateway: server-side model selection, fallback, cost/latency tracking, and versioning.
- Observability: trace user action → agent run → retrieval → tool calls → approval → mutation.
- Evaluation Harness: common tests for every agent, skill, plugin, major prompt, and model change.

6.2 Repository Structure

- /platform/agents
- /platform/skills
- /platform/plugins
- /platform/mcp
- /platform/policies
- /platform/schemas
- /platform/evals
- /platform/observability
- /platform/knowledge
- /platform/deploy

6.3 Knowledge/RAG

- Ingestion → validation/classification → chunking → embedding → vector index → retrieval → reranking → context assembly → agent.

- Every retrieved item carries source, version, tenant/scope, document status, and provenance.
- Only approved/eligible knowledge is promoted to shared organizational knowledge.
- Store source documents in object storage and metadata/provenance in Postgres.
- Use pgvector initially rather than a separate vector platform.
- Defer knowledge graph implementation until a demonstrated relationship-query use case warrants it.

6.4 Agent Memory

- Working memory: short-lived context for one run/session.
- Episodic memory: engagement events, decisions, outcomes, and interaction summaries.
- Semantic/shared memory: approved organizational knowledge retrieved through RAG.
- Memory is scoped by organization, engagement, user permissions, and classification. Agents must not silently promote arbitrary conversation content into durable shared memory.

6.5 Evaluation & Quality Gates

Level	Check	Applies to	Gate
Q0	Unit/schema/permission tests	Code, tools, RLS, structured outputs	Required
Q1	Golden-set regression	Agents, prompts, RAG, workflows	Required
Q2	LLM-as-a-judge	Quality, relevance, completeness, groundedness	Required; calibrated
Q3	Adversarial/safety	Injection, tool abuse, data leakage	Required
Q4	Human acceptance	High-impact workflows and UX	Required pre-production
Q5	Production monitoring	Drift, errors, latency, cost, feedback	Continuous

LLM judges are not the sole authority for high-impact actions. Deterministic checks, permission checks, policy checks, and human approval remain authoritative where required.

6.6 Deployment

- Frontend is separate from the server-side agent runtime.
- Model credentials, MCP credentials, service keys, and integration secrets are server-side only.
- Agent workers have timeouts, retries, rate limits, and bounded execution.
- Use environment-specific configuration and secrets management.
- CI/CD runs unit tests, schema/security checks, evaluation suites, and migration checks before production.
- Use staging with representative non-sensitive evaluation data.
- Support rollback of application, prompt, policy, and model configuration versions.
- Monitor cost, latency, errors, tool failures, retrieval quality, and evaluation regressions.

7. Multi-Agent Responsibility Matrix

Agent	Primary responsibility	Shared capabilities	Does not own
Scout	Qualification, readiness, routing	RAG, skills, MCPs, model gateway, approvals	Tasks, calendar, CRM persistence
Architect	Assessment, planning, scoping	RAG, skills, MCPs, contract service, approvals	Independent task/calendar stores
Chronicle	Impact synthesis, lessons, knowledge candidates	RAG, memory, evaluation, approvals	Operational CRM state

8. Option B — In-App Contract Signing

1. Generate draft engagement charter from structured engagement data.

2. Present preview with source data and permission-controlled edits.
3. Authorized representative reviews the final document in-app.
4. Signer explicitly confirms intent and signs.
5. Backend records signer identity, timestamp, document version/hash, and signature event.
6. Backend verifies signer authorization and exact signed document version.
7. Successful signature creates the authoritative lifecycle transition.
8. Executed artifact and audit trail remain accessible.

9. Firebase → Supabase Migration

Recommendation: switch Success Portal to Supabase now. The shared GrantPilot + Success Portal identity requirement makes a single Supabase Auth/user foundation preferable to maintaining Firebase plus Supabase and synchronizing identities and permissions.

9. Define canonical Supabase schema, roles, organizations, memberships, and RLS.
10. Connect Success Portal to the same Supabase Auth project used by GrantPilot.
11. Migrate Firebase identities/data using a temporary identity mapping and reconciliation.
12. Run shadow/dual-read verification where useful; avoid long-lived dual-write unless required.
13. Cut production reads/writes to Supabase.
14. Decommission Firebase after reconciliation, monitoring, and a defined rollback window.

10. Delivery Plan

Part	Workstream	Primary deliverables
1	UI & UX	Auth shell, dashboards, Scout review, Architect workflow, contract preview/signing, engagement/task views, accessibility.
2	CRM & Data Engineering	Supabase schema/Auth/RLS, lifecycle entities, audit trail, tasks, approvals, contracts/signatures, Firebase migration.
3	Knowledge/RAG, Agent Memory & Deployment	Skills/plugins/MCP registries, agent runtime, RAG, memory, evaluation harness, observability, CI/CD, production deployment.

11. Implementation Review Checklist

- ☐ Shared Supabase project confirmed with GrantPilot.
- ☐ Identity, organization, membership, and RLS model approved.
- ☐ Capability ownership matrix approved.
- ☐ Agent/skill/plugin/MCP boundaries approved.
- ☐ Structured schemas and policy contracts versioned.
- ☐ Option B contract-signing flow approved.
- ☐ Firebase migration and rollback plan approved.
- ☐ Golden evaluation datasets identified.
- ☐ LLM judge rubric calibrated against human labels.
- ☐ Adversarial test suite defined.
- ☐ Production observability and audit requirements approved.
- ☐ MVP explicitly excludes unnecessary knowledge-graph/autonomous-agent complexity.

12. Open Decisions

- Confirm exact Supabase project that becomes the shared identity root.
- Confirm roles and tenant/organization model.
- Confirm legal/e-sign requirements and whether a third-party signature provider is ultimately needed.
- Confirm MVP external integrations versus post-MVP plugins.
- Confirm initial model providers and evaluation judge models.
- Confirm retention/deletion policy.
- Confirm production hosting/runtime for server-side agent execution.

Prepared for Implementation Review